Programmation Orientée Objet (POO) à destination de la robotique

Travaux Pratique : Navigation réactive

Enseignant : Adam GOUGUET
Mail : adam.gouguet@imt-nord-europe.fr
Année : 2025–2026

Table des matières

1 Objectifs du TP **1**

2 Prérequis **2**

3 Principe de la navigation réactive **2**

4 Pourquoi utiliser un design pattern ? **2**

5 Design Pattern Strategy **3**

6 Architecture demandée **3**

7 Travail demandé **4**
7.1 Contrôleur 1 - Évitement simple 4
7.2 Contrôleur 2 - Évitement directionnel 4
7.3 Contrôleur 3 - Navigation vers objectif 4
7.4 Dans le main : tester vos stratégies 5

8 Questions d'analyse **6**

9 Pour aller plus loin... **6**

10 Conclusion **6**

1 Objectifs du TP

Ce TP a pour objectif d'implémenter une navigation autonome réactive pour un robot mobile simulé.

À la fin du TP, vous devrez être capables de :

— comprendre le principe de la navigation réactive,
— exploiter des données capteurs pour prendre une décision,
— implémenter un contrôleur comportemental,

— structurer votre code avec une architecture orientée objet simple,
— analyser le comportement d'un robot autonome.

2 Prérequis

Vous devez avoir réalisé le TP sur les capteurs, en particulier :
— comprendre le fonctionnement du Lidar simulé,
— savoir lire un vecteur de distances,
— comprendre la normalisation des mesures.
Si ce n'est pas le cas, faites ce TP avant de continuer.

3 Principe de la navigation réactive

Une navigation réactive signifie que le robot décide de ses actions uniquement à partir de ses capteurs instantanés, sans carte globale ni planification de trajectoire.

Cycle de contrôle :

```
perception -> décision -> action
```

À chaque itération :
— le robot lit ses capteurs,
— le contrôleur décide d'une commande,
— le robot applique cette commande.
Aucune mémoire du passé ni connaissance de l'environnement n'est utilisée.

4 Pourquoi utiliser un design pattern ?

En robotique, on teste souvent plusieurs comportements :
— éviter obstacles
— suivre un mur
— aller vers une cible
— explorer
Sans architecture adaptée, on finirait avec un code du type :

```
if mode == 1:
    comportement1()
elif mode == 2:
    comportement2()
elif mode == 3:
    comportement3()
```

Ce code est :
— difficile à maintenir
— difficile à étendre
— peu lisible

5 Design Pattern Strategy

Le pattern Strategy consiste à encapsuler des comportements interchangeables dans des classes séparées. Au lieu d'un gros bloc de conditions, on crée une interface commune :

```
class Strategy:
    def compute_command(self, observation):
        pass
```

Puis plusieurs implémentations :

```
class AvoidStrategy(Strategy):
class FreeDirectionStrategy(Strategy):
class GoalStrategy(Strategy):
```

Le robot utilise simplement :

```
cmd = strategy.compute_command(obs)
```

Il ne sait même pas quelle stratégie il utilise. C'est exactement ce qu'on veut : robot indépendant des comportements.

6 Architecture demandée

Vous devez implémenter trois niveaux distincts :

1 - Robot (vous l'avez déjà)

Il ne contient aucune intelligence.
Responsabilité :
— lire capteurs
— appliquer commandes

2 - Navigator (Context)

C'est lui qui utilise une stratégie.

```python
class Navigator:

    def __init__(self, strategy):
        self.strategy = strategy

    def set_strategy(self, strategy):
        self.strategy = strategy

    def step(self, observation):
        return self.strategy.compute_command(observation)
```

Le Navigator :
— ne connaît pas l'algorithme

— ne connaît pas le type de stratégie
— délègue la décision

> Cela permet de changer de strategie dynamiquement si besoin.

3 - Strategy (interface)

```
class Strategy:
    def compute_command(self, observation):
        raise NotImplementedError
```

Toutes les stratégies doivent hériter de cette classe.

7 Travail demandé

Vous devez implémenter successivement plusieurs stratégies de navigation.

7.1 Contrôleur 1 - Évitement simple

Objectif : empêcher les collisions.

```
si obstacle proche devant:
    tourner
sinon:
    avancer
```

> Cela vous demande de comprendre et d'utiliser le LiDAR.

7.2 Contrôleur 2 - Évitement directionnel

Le robot doit :
— détecter de quel côté l'espace est le plus libre
— tourner vers ce côté

> Comparez la moyenne des distances gauche/droite.

7.3 Contrôleur 3 - Navigation vers objectif

On ajoute une position cible.
Le robot doit :
— se diriger vers la cible
— éviter les obstacles en même temps

> 💡 Essayez de combiner deux comportements : attraction vers la cible et répulsion des obstacles

7.4 Dans le main : tester vos stratégies

Une fois vos stratégies implémentées, vous devez pouvoir les tester facilement sans modifier le robot ni le Navigator. L'objectif est simplement de changer l'objet stratégie.

1) Création des objets principaux

```
robot = RobotMobile(...)
lidar = Lidar(robot)
robot.ajouter_capteur(lidar)

strategy = AvoidStrategy()
navigator = Navigator(strategy)
```

Le Navigator reçoit une stratégie au constructeur. Il n'a aucune connaissance de son fonctionnement interne.

2) Boucle de simulation

À chaque itération :
- on lit les capteurs
- on demande une commande au Navigator
- on applique la commande

```
while simulation_active:

    observation = robot.get_observation()

    cmd = navigator.step(observation)

    robot.commander(**cmd)
```

3) Changer de stratégie dynamiquement

Vous pouvez changer de comportement sans modifier la boucle :

```
navigator.set_strategy(FreeDirectionStrategy())
```

ou

```
navigator.set_strategy(GoalStrategy(goal))
```

Cela fonctionne car toutes les stratégies respectent la même interface.

4) Tester plusieurs comportements

Exemple de séquence de test :

1. lancer avec AvoidStrategy
2. observer le comportement
3. changer pour FreeDirectionStrategy
4. comparer

Vous devez pouvoir tester différents comportements simplement en remplaçant une ligne.

Résumé

Le rôle de chaque composant :
— Robot : exécute les commandes
— Navigator : choisit la stratégie
— Strategy : décide de l'action
La boucle principale ne change jamais. Seule la stratégie change.

8 Questions d'analyse

En faisant plusieurs test dans la simulation essayez de répondre à ces questions :

1. Pourquoi la navigation réactive peut-elle échouer dans certains environnements ?
2. Donnez un exemple de situation piège.
3. Quels sont les avantages de cette approche ?
4. Quelle amélioration architecturale permettrait de résoudre ces limites ?

9 Pour aller plus loin...

Ce TP introduit volontairement une architecture minimale.
Pour la suite, vous ajouterez :
— un planificateur global,
— un planificateur local,
— une carte de l'environnement.
Vous devrez alors faire évoluer votre architecture actuelle sans tout réécrire.

10 Conclusion

Ce TP constitue la première étape vers une architecture complète de navigation robotique. L'objectif n'est pas seulement de faire fonctionner le robot, mais de comprendre les limites d'une approche purement réactive.

Ces limites motiveront l'introduction de planificateurs plus avancés dans les prochains travaux pratiques.